

Avaliacao do GPT-OSS-20B com Chain-of-Thought e Process Supervision Promptado

Evaluation harness local com vLLM

Rodadas principais: 20260510-185218 e 20260510-231913

Abstract

Este relatorio consolida duas avaliacoes locais do modelo `openai/gpt-oss-20b`. A primeira compara prompting few-shot padrao contra few-shot com *chain-of-thought* (CoT), inspirada em *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. A segunda implementa uma aproximacao promptada de *Process Supervision*, inspirada em *Let's Verify Step by Step*, na qual o mesmo modelo gera solucoes para problemas MATH e tambem julga cada passo via logprobs. Todos os experimentos foram executados localmente com vLLM em quatro GPUs RTX 4000 Ada. Os resultados indicam que CoT melhora GSM8K de 94,5% para 96,5%, mas nao ajuda em tarefas simbolicas simples. No experimento de Process Supervision, o best-of-8 melhora a acuracia de 86,0% para 96,0%, mas o PRM promptado empata com majority vote e nao supera o baseline simples. Todo o codigo usado para gerar, avaliar e documentar os experimentos esta disponivel no repositorio publico do projeto.

1 Configuracao experimental

As avaliacoes usaram o modelo `openai/gpt-oss-20b` servido localmente por vLLM com API OpenAI-compatible. O servidor foi executado em quatro GPUs NVIDIA RTX 4000 Ada Generation com tensor parallel igual a 4. O objetivo nao foi reproduzir todos os numeros dos papers originais, mas implementar experimentos locais controlados que permitissem estudar dois mecanismos: prompting com CoT e selecao por supervisao de processo.

Table 1: Configuracao geral dos experimentos.

Campo	Valor
Modelo	<code>openai/gpt-oss-20b</code>
Backend	vLLM OpenAI-compatible local
Paralelismo	Tensor parallel em 4 GPUs
GPUs	4x NVIDIA RTX 4000 Ada Generation
Hardware alvo	Inferencia local distribuida
Raw CoT interno	Nao utilizado no relatorio
Artefatos CoT	<code>experiments/01_chain_of_thought/artifacts/20260510-185218/</code>
Artefatos PRM	<code>experiments/02_process_supervision/artifacts/20260510-231913/</code>

Em ambos os estudos, a avaliacao se baseia apenas em respostas visiveis, respostas parseadas e metricas agregadas. O relatorio nao usa nem expande raw chain-of-thought interno do modelo.

2 Codigo, organizacao e artefatos

Todo o codigo-fonte, a documentacao de reproducao e os artefatos resumidos deste trabalho estao disponiveis no repositorio GitHub [5]: <https://github.com/costadev00/gpt-oss-eval-chain-of-thought>. O repositorio foi organizado para separar claramente os dois experimentos: `experiments/01_chain_of_thought/` contem a avaliacao de CoT, e `experiments/02_process_supervision/` contem o teste de Process Supervision promptado. O arquivo `README.md` funciona como guia de execucao, enquanto este artigo consolida a metodologia, os resultados e a analise critica.

Table 2: Principais arquivos do repositorio usados neste estudo.

Arquivo ou pasta	Papel no projeto
<code>README.md</code>	Guia principal em portugues, com instalacao, comandos, metodologia resumida, hardware e resultados.
<code>pyproject.toml</code>	Metadados do pacote Python, dependencias e configuracao de testes.
<code>cot_eval/run.py</code>	Ponto de entrada do experimento CoT versus resposta direta.
<code>cot_eval/prompts.py</code>	Prompts e contrato de saida usados nas condicoes <i>standard</i> e CoT.
<code>cot_eval/tasks.py</code>	Carregamento de GSM8K e geracao deterministica das tarefas Last Letter e Coin Flip.
<code>cot_eval/audit_parser.py</code>	Auditoria offline do parser sem nova chamada ao modelo.
<code>cot_eval/prm_run.py</code>	Ponto de entrada do experimento de Process Supervision promptado.
<code>cot_eval/math_tasks.py</code>	Carregamento e normalizacao numerica do benchmark MATH.
<code>cot_eval/prm_scoring.py</code>	Divisao de solucoes em passos, calculo de scores PRM e majority vote.
<code>experiments/01_chain_of_thought/</code>	README proprio e artefatos preservados da rodada principal de CoT.
<code>experiments/02_process_supervision/</code>	README proprio e artefatos preservados da rodada principal de Process Supervision.
<code>reports/gpt_oss_20b_cot_prm_report.tex</code>	Fonte deste artigo consolidado.

As saidas brutas completas, como `predictions.jsonl`, `candidates.jsonl`, `selected.jsonl` e `step_scores.jsonl`, nao sao versionadas no repositorio enxuto porque podem ser regeneradas pelos comandos documentados. Para manter rastreabilidade sem inflar o repositorio, preservamos os arquivos pequenos de configuracao, metricas e analise nas pastas de cada experimento.

3 Experimento 1: CoT versus resposta direta

3.1 Metodo

O primeiro experimento compara duas condicoes de prompting. A condicao *Sem CoT* usa exemplos few-shot com resposta direta. A condicao *CoT* usa exemplos few-shot com raciocinio intermediario antes da resposta final. As duas condicoes compartilham o mesmo modelo, backend, temperatura

zero, `reasoning_effort=medium` e o mesmo contrato de saída: `Final answer: <answer>`. Esse contrato reduz a chance de o parser extrair numeros intermediarios em vez da resposta final.

A implementacao deste experimento esta concentrada em `cot_eval/run.py`. Os prompts ficam em `cot_eval/prompts.py`, as tarefas em `cot_eval/tasks.py` e as funcoes de extracao/comparacao de respostas em `cot_eval/scoring.py`. Os artefatos preservados da rodada principal ficam em `experiments/01_chain_of_thought/artifacts/20260510-185218/`.

Foram avaliadas tres tarefas: GSM8K, Last Letter e Coin Flip. GSM8K representa raciocinio aritmetico multi-etapa; Last Letter e uma tarefa simbolica local; e Coin Flip avalia rastreamento simples de estado binario.

GSM8K e um benchmark de problemas matematicos em linguagem natural criado pela OpenAI para avaliar raciocinio aritmetico multi-etapa em modelos de linguagem. O nome vem de “Grade School Math 8K”, pois o conjunto contem cerca de 8,5 mil problemas no estilo matematica de ensino fundamental, normalmente resolviveis com operacoes como soma, subtracao, multiplicacao, divisao, proporcoes e contagem. Cada exemplo traz um enunciado textual e uma solucao com resposta final numerica. Um problema tipico seria: “Joao tinha 12 macas, comprou mais 8 e depois dividiu igualmente entre 5 amigos; com quantas macas cada amigo ficou?”, exigindo que o modelo decomponha o texto em etapas aritmeticas antes de chegar a resposta. No nosso experimento, usamos o dataset publico `openai/gsm8k`, disponivel no Hugging Face, especificamente a configuracao `main` e o split `test`. Do conjunto completo de aproximadamente 8,5 mil problemas, avaliamos uma amostra de 200 problemas do split de teste. A resposta correta foi extraida do campo de solucao do dataset, onde o valor final aparece apos o marcador `####`, e o modelo foi considerado correto quando sua linha `Final answer: <answer>` correspondia numericamente a esse valor.

A tarefa Last Letter foi usada como uma avaliacao simbolica simples, inspirada nas tarefas sinteticas utilizadas no artigo de Chain-of-Thought. Nessa tarefa, o modelo recebe uma sequencia de palavras ou nomes e deve retornar a concatenacao da ultima letra de cada termo, preservando a ordem em que os termos aparecem. Por exemplo, para a entrada “Donald Hamilton”, a resposta esperada e `dn`, pois a ultima letra de “Donald” e `d` e a ultima letra de “Hamilton” e `n`. Diferentemente do GSM8K, essa tarefa nao vem de um dataset externo fixo: ela foi gerada localmente pelo nosso harness, de forma deterministica a partir de uma seed, para permitir controle total sobre os exemplos e as respostas ouro. No experimento, avaliamos 100 exemplos de Last Letter. A resposta do modelo foi extraida da linha `Final answer: <answer>` e comparada exatamente, apos normalizacao textual, com a string ouro gerada pela regra da tarefa.

A tarefa Coin Flip tambem foi gerada localmente e segue o estilo de tarefas simbolicas de rastreamento de estado usadas para avaliar Chain-of-Thought. Nessa tarefa, o enunciado descreve o estado inicial de uma moeda e uma sequencia de acoes, como virar ou nao virar a moeda. O modelo deve determinar se, ao final da sequencia, a moeda esta com a face inicial para cima, respondendo em formato `yes` ou `no`. Um exemplo simplificado seria: “A coin is heads up. Ana flips the coin. Ana flips the coin again. Is the coin still heads up?”; como duas viradas retornam a moeda ao estado inicial, a resposta correta seria `yes`. No nosso experimento, avaliamos 100 exemplos de Coin Flip, tambem gerados deterministicamente por seed. A resposta ouro foi calculada por regra, simulando as mudancas de estado da moeda, e a saida do modelo foi avaliada por um parser de respostas `yes/no`.

Table 3: Tarefas usadas no experimento CoT.

Tarefa	Origem	Descricao
GSM8K	openai/gsm8k, split test, config main	Conjunto de problemas matematicos de ensino fundamental. Cada item contem um enunciado textual e uma resposta numerica final. A avaliacao usa 200 exemplos e extrai o <i>gold</i> do campo de resposta apos o marcador ####.
Last Letter	Gerada localmente, inspirada no paper de CoT	Tarefa simbolica em que o modelo recebe uma lista de palavras ou nomes e deve concatenar a ultima letra de cada termo. Por exemplo, para dois nomes, a saida esperada e uma string com duas letras. A rodada usa 100 exemplos gerados de forma deterministica por seed.
Coin Flip	Gerada localmente, inspirada no paper de CoT	Tarefa de rastreamento de estado. O enunciado descreve uma moeda inicialmente em um estado e uma sequencia de acoes de virar ou nao virar a moeda. O modelo deve responder se a moeda termina com a face inicial para cima, em formato <i>yes/no</i> . A rodada usa 100 exemplos gerados de forma deterministica por seed.

3.2 Resultados

Table 4: Acuracia por tarefa e condicao no experimento CoT.

Tarefa	Condicao	Corretas/N	Acuracia	Delta CoT
GSM8K	Sem CoT	189/200	94,5%	–
GSM8K	CoT	193/200	96,5%	+2,0 p.p.
Last Letter	Sem CoT	85/100	85,0%	–
Last Letter	CoT	84/100	84,0%	-1,0 p.p.
Coin Flip	Sem CoT	100/100	100,0%	–
Coin Flip	CoT	100/100	100,0%	+0,0 p.p.

3.3 Analise critica

O principal ganho de CoT aparece em GSM8K: a acuracia sobe de 94,5% para 96,5%. A comparacao pareada mostrou que CoT corrigiu seis itens que a resposta direta errou, enquanto perdeu dois itens que a resposta direta acertou. Esse padrao e consistente com a hipotese de que demonstracoes intermediarias ajudam em problemas aritmeticos que exigem decomposicao. Ainda assim, o ganho e pequeno e a amostra nao e suficiente para afirmar uma vantagem universal.

Em Last Letter, CoT nao melhorou o desempenho. A tarefa exige uma operacao simbolica curta e exata; nesse caso, raciocinio intermediario pode apenas criar mais pontos de erro. Em Coin Flip, ambas as condicoes atingiram 100%, o que torna a tarefa pouco informativa para distinguir os metodos nesta configuracao.

Table 5: Custo medio por item no experimento CoT.

Tarefa/condicao	Latencia (s)	Prompt tok.	Resposta tok.	Total tok.
GSM8K Sem CoT	1,03	584,4	202,1	786,5
GSM8K CoT	1,12	892,4	219,0	1111,4
Last Letter Sem CoT	0,38	303,4	72,8	376,1
Last Letter CoT	0,75	428,4	146,7	575,1
Coin Flip Sem CoT	1,51	502,3	298,2	800,5
Coin Flip CoT	0,82	852,3	158,4	1010,8

Do ponto de vista de custo, CoT aumenta o numero medio de tokens. Em GSM8K, o total medio cresce de 786,5 para 1111,4 tokens por item, aumento aproximado de 41%. Assim, o uso de CoT parece justificado para raciocinio matematico multi-etapa, mas nao para tarefas simbolicas simples ou ja saturadas.

4 Experimento 2: *Process Supervision* promptado

4.1 Metodo

O segundo experimento implementa uma aproximacao promptada de Process Supervision. O GPT-OSS-20B gera oito solucoes por problema MATH. Em seguida, o mesmo modelo julga cada passo de cada solucao com os labels **positive**, **neutral** e **negative**. As probabilidades sao estimadas com logprobs do endpoint de completions. Seguindo a configuracao principal descrita no paper de referencia, **neutral** e tratado como positivo, e o score da solucao e o produto dos scores de seus passos.

A implementacao deste estudo esta em `cot_eval/prm_run.py`. O carregamento e a normalizacao do MATH ficam em `cot_eval/math_tasks.py`; os prompts de geracao e julgamento ficam em `cot_eval/prm_prompts.py`; e a divisao em passos, o calculo dos scores e o majority vote ficam em `cot_eval/prm_scoring.py`. Os artefatos preservados da rodada principal ficam em `experiments/02_process_supervision/artifacts/20260510-231913/`.

Este experimento nao treina um PRM real e nao usa labels humanos de PRM800K. Ele mede a utilidade operacional de um verificador promptado com o mesmo modelo que gerou as solucoes. Isso cria risco de erros correlacionados: uma solucao errada, mas plausivel para o gerador, tambem pode parecer plausivel para o verificador.

O benchmark usado foi o MATH, tambem conhecido nesta implementacao pelo dataset `EleutherAI/hendrycks` no Hugging Face. Esse conjunto foi criado para avaliar resolucao de problemas matematicos mais dificeis que GSM8K, em estilo olimpiada, competicao escolar e cursos avancados de ensino medio. Os enunciados cobrem areas como algebra, geometria, teoria dos numeros, precalculo, contagem e probabilidade, e normalmente exigem uma sequencia de passos matematicos, nao apenas uma operacao aritmetica direta. Um exemplo tipico de problema MATH pode pedir, por exemplo, a simplificacao de uma expressao algebrica, a area de uma figura geometrica ou a solucao de uma equacao com restricoes. No nosso experimento, usamos apenas o split `test` e quatro configuracoes do dataset: `intermediate_algebra`, `precalculus`, `geometry` e `number_theory`. Esses subconjuntos somaram 2468 itens vistos pelo loader. Como a avaliacao automatica dependia de comparacao numerica, filtramos os exemplos para manter apenas aqueles em que a resposta final numerica podia ser extraida de uma expressao `\boxed{...}`; nessa etapa, 781 itens foram descartados. A partir dos itens restantes, selecionamos 50 problemas usando a seed 43. Para cada problema, o modelo

gerou 8 solucoes candidatas, totalizando 400 solucoes avaliadas pelo seletor PRM, majority vote e oracle.

Antes da rodada final, o experimento passou por uma etapa incremental de implementacao e validacao. Primeiro, foi implementado um pipeline minimo para gerar solucoes, quebrar cada solucao em passos, avaliar cada passo com um verificador promptado e salvar os artefatos em JSONL, CSV, e Markdown. Em seguida, rodamos testes menores, incluindo uma rodada inicial com poucos problemas e uma rodada 10x4, para validar o formato das respostas, a escrita dos arquivos, a agregacao das metricas e o comportamento dos seletores *first*, *majority vote*, *PRM best-of-N* e *oracle best-of-N*. Essas rodadas preliminares tambem revelaram problemas praticos importantes, especialmente truncamento de respostas e erros de parsing de fracoes finais. Por isso, antes da rodada 50x8, aumentamos o orcamento de geracao para `max_tokens=4096`, ajustamos a temperatura para 0,9 a fim de aumentar a diversidade dos candidatos, e corrigimos o parser para reconhecer respostas como $\frac{1}{3}$, $\frac{5}{9}$ e expressoes com `\displaystyle`. Assim, os resultados reportados representam a rodada final apos a estabilizacao do pipeline, nao apenas uma execucao unica isolada.

Table 6: Configuracao da rodada PRM.

Campo	Valor
Benchmark	EleutherAI/hendrycks_math
Configs MATH	intermediate algebra, precalculus, geometry, number theory
Filtro	Apenas respostas numericas extraiveis
Problemas	50
Solucoes por problema	8
Candidatos totais	400
Temperatura de geracao	0,9
<code>max_tokens</code>	4096
Seed	43
Scoring PRM	Produto dos scores por passo

O protocolo de selecao foi inspirado no desenho experimental de *Let’s Verify Step by Step* [2], que avalia nao apenas uma unica resposta do modelo, mas tambem estrategias para selecionar a melhor solucao entre varias amostras. Por isso, comparamos quatro metodos. O primeiro, *first sample*, usa simplesmente a primeira solucao gerada e serve como baseline sem busca. O segundo, *majority vote*, escolhe a resposta final mais frequente entre as oito solucoes candidatas; esse baseline é importante porque muitos ganhos de *best-of-N* podem vir apenas da agregacao de varias amostras, sem qualquer verificador por passo. O terceiro, *PRM best-of-N*, seleciona a solucao com maior score de processo, isto é, a solucao cujos passos foram julgados como mais confiaveis pelo verificador promptado. Por fim, *oracle best-of-N* escolhe uma solucao correta sempre que ela existe entre as candidatas; esse metodo nao é utilizavel na pratica, mas funciona como teto superior para medir quanta margem de melhoria ainda existiria se o seletor fosse perfeito.

4.2 Resultados

4.3 Analise critica

O resultado mais importante é que gerar oito solucoes por problema melhora substancialmente o desempenho: o primeiro candidato acerta 86,0%, enquanto majority vote acerta 96,0%. Isso mostra que o modelo frequentemente contem a resposta correta no conjunto de candidatos mesmo quando

Table 7: Acuracia corrigida por metodo de selecao no experimento PRM.

Metodo	Corretas/N	Acuracia	Falhas parse	Cobertura parse
First sample	43/50	86,0%	5	90,0%
Majority vote	48/50	96,0%	0	100,0%
PRM best-of-N	48/50	96,0%	0	100,0%
Oracle best-of-N	49/50	98,0%	1	98,0%

Table 8: Custo medio por problema selecionado no experimento PRM.

Metodo	Lat. geracao (s)	Lat. PRM (s)	Tok. geracao	Tok. PRM
First sample	7,94	0,00	1794,4	0,0
Majority vote	61,67	0,00	14017,2	0,0
PRM best-of-N	61,67	2,14	14017,2	39707,4
Oracle best-of-N	61,67	0,00	14017,2	0,0

a primeira amostra falha.

Entretanto, o PRM promptado nao supera majority vote. Apos a correcao do parser, os dois metodos empatam em 48/50. O oracle chega a 49/50, mostrando que havia uma oportunidade adicional de selecao que nem majority vote nem PRM capturaram. O caso principal foi um problema de precalculo em que havia apenas um candidato correto entre oito; tanto a votacao quanto o PRM escolheram uma resposta incorreta. Esse caso ilustra a limitacao do verificador promptado: quando a solucao correta e rara e longa, o score por produto pode penaliza-la.

O diagnostico de scores reforca essa cautela. Embora o score medio de candidatos corretos seja maior que o de candidatos incorretos em alguns resumos, a ordenacao fina nao e confiavel: a AUC do score PRM para separar candidatos corretos de incorretos ficou em aproximadamente 0,397. Isso indica que o score bruto nao deve ser interpretado como probabilidade calibrada de correcao. Uma causa provavel e o uso do produto dos scores por passo, que tende a favorecer solucoes mais curtas. De fato, o PRM selecionou solucoes com media de 8,86 passos, enquanto majority vote selecionou solucoes com media de 14,06 passos.

O custo operacional do PRM promptado tambem e relevante. Majority vote exige apenas os tokens de geracao dos oito candidatos. PRM best-of-N exige os mesmos tokens de geracao e ainda adiciona cerca de 39707 tokens de verificacao por problema. Portanto, nesta configuracao, o PRM promptado aumenta muito o custo sem melhorar a acuracia sobre majority vote.

5 Discussao geral

Os dois experimentos devem ser lidos como uma demonstracao didatica da evolucao recente em modelos de linguagem: primeiro, observa-se que induzir o modelo a explicitar passos intermediarios por meio de *chain-of-thought* pode melhorar respostas em tarefas que exigem raciocinio; em seguida, esses passos intermediarios passam a ser tratados nao apenas como texto auxiliar, mas como objetos que podem ser avaliados, comparados e usados como sinal de treinamento. No Experimento 1, a melhoria em GSM8K mostra o fenomeno original do CoT: mesmo sem alterar os pesos do modelo, mudar o formato do prompt desloca o comportamento do modelo para solucoes mais decompostas e frequentemente mais corretas. Esse resultado e especialmente claro em problemas aritmeticos multi-etapa, nos quais a resposta direta pode falhar por pular uma etapa de calculo ou interpretar

Table 9: Diagnostico dos candidatos gerados no experimento PRM.

Metrica	Valor
Candidatos totais	400
Candidatos corretos apos correcao do parser	345
Falhas de parse em candidatos	39
Problemas com 8/8 candidatos corretos	36
Problemas com mistura de candidatos corretos e incorretos	13
Problemas sem nenhum candidato correto	1
Problemas com uma unica resposta parseavel distinta	43
AUC do score PRM para separar candidato correto/incorreto	0,397

mal uma relacao do enunciado.

O Experimento 2 representa o passo conceitual seguinte. A partir do momento em que uma solucao passa a ser escrita como uma sequencia de passos, torna-se possivel avaliar o processo, e nao apenas a resposta final. Essa e a motivacao central de Process Supervision: transformar o raciocinio intermediario em um sinal de supervisao para selecionar melhores solucoes e, em sistemas maiores, alimentar etapas de post-training e pipelines de aprendizado por reforco. Nesta implementacao, nao treinamos um PRM real; usamos um PRM promptado para mostrar operacionalmente como esse sinal poderia ser produzido. Mesmo assim, o experimento ilustra a arquitetura da ideia: gerar varias solucoes, decompor em passos, julgar cada passo, agregar os scores e escolher uma solucao por best-of-N.

Os resultados tambem mostram por que esse tipo de supervisao se tornou importante. No experimento PRM, o conjunto de oito candidatos por problema frequentemente continha uma resposta correta mesmo quando a primeira amostra falhava. Isso evidencia que parte do problema nao e apenas “o modelo sabe ou nao sabe”, mas tambem como selecionar, entre varias trajetorias possiveis, a trajetoria mais confiavel. O majority vote ja foi um baseline forte nesta rodada, e o PRM promptado nao o superou; ainda assim, essa limitacao e informativa. Ela mostra que simplesmente pedir ao mesmo modelo para julgar seus proprios passos nao equivale a ter um reward model treinado com supervisao humana. Em outras palavras, o experimento reproduz em pequena escala a motivacao para PRMs treinados: e necessario transformar avaliacoes de processo em um sinal mais calibrado, robusto e util para selecao e treinamento.

Assim, o valor principal do projeto nao esta apenas nas acuracias finais, mas na cadeia experimental que elas tornam visivel. CoT demonstra que a forma de externalizar o raciocinio altera o desempenho do modelo. Process Supervision mostra como esse raciocinio externalizado pode ser convertido em dados de avaliacao por passo. Essa passagem, de “promptar para raciocinar” para “supervisionar o processo de raciocinio”, e uma das bases conceituais que influenciaram novas formas de post-training, reward modeling e aprendizado por reforco em modelos de linguagem.

6 Limitacoes

Este estudo tem quatro limitacoes principais. Primeiro, as amostras sao exploratorias: 200 itens de GSM8K, 100 itens por tarefa simbolica e 50 problemas MATH no PRM. Segundo, o PRM e promptado, nao treinado com labels humanos; logo, nao deve ser confundido com um reward model supervisionado real. Terceiro, o mesmo modelo gera e julga, aumentando o risco de erros correlacionados. Quarto, o filtro numeric-only simplifica MATH e remove exemplos cujo formato

de resposta nao era facilmente comparavel por regra.

7 Conclusao

Para o GPT-OSS-20B executado localmente, CoT mostrou beneficio moderado em GSM8K, mas nao trouxe ganho em tarefas simbolicas simples. No estudo de Process Supervision, o best-of-8 foi efetivo, mas o PRM promptado nao superou majority vote e adicionou custo substancial. Assim, os resultados sustentam uma recomendacao pragmatica: usar CoT seletivamente para problemas matematicos multi-etapa e usar best-of-N com majority vote como baseline forte antes de adotar verificadores por passo. Para demonstrar ganho real de Process Supervision, o proximo passo deveria comparar agregadores alternativos de score por passo, como media de log-score ou minimo por passo, ou treinar um PRM real com labels humanos.

Todos os scripts usados para executar essas avaliacoes, os testes automatizados, os artefatos resumidos e o README de reproducao estao disponiveis no repositorio publico do projeto [5]: <https://github.com/costadev00/gpt-oss-eval-chain-of-thought>.

References

- [1] Jason Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903, 2022.
- [2] Hunter Lightman et al. Let’s Verify Step by Step. arXiv:2305.20050, 2023.
- [3] OpenAI. GSM8K dataset. <https://huggingface.co/datasets/openai/gsm8k>.
- [4] EleutherAI. Hendrycks MATH dataset. https://huggingface.co/datasets/EleutherAI/hendrycks_math.
- [5] CostaDev00. GPT-OSS eval chain-of-thought repository. <https://github.com/costadev00/gpt-oss-eval-chain-of-thought>.